



UNIVERSIDADE FEDERAL RURAL DO SEMI-ÁRIDO
BACHARELADO INTERDISCIPLINAR EM TECNOLOGIA DA INFORMAÇÃO
LABORATÓRIO DE ESTRUTURA DE DADOS II
KENNEDY REURISON LOPES

JOSÉ EDUARDO FREITAS OLIVEIRA
REUBISMAR GOMES DOS SANTOS JUNIOR
RUAN DANTAS BANDEIRA

RELATÓRIO DE LAB. ESTRUTURA DE DADOS II
PROJETO: SISTEMA DE CONSULTA EFICIENTE COM TABELA HASH E
FILTRO DE BLOOM

PAU DOS FERROS - RN

2026

JOSÉ EDUARDO FREITAS OLIVEIRA

jose.oliveira94303@alunos.ufersa.edu.br

REUBISMAR GOMES DOS SANTOS JUNIOR

reubismar.junior@alunos.ufersa.edu.br

RUAN DANTAS BANDEIRA

ruan.bandeira@alunos.ufersa.edu.br

RELATÓRIO DE LAB. ESTRUTURA DE DADOS II
PROJETO: SISTEMA DE CONSULTA EFICIENTE COM TABELA HASH E
FILTRO DE BLOOM

Relatório de atividades apresentado como requisito parcial para aprovação na disciplina de Laboratório de Estrutura de Dados II, do curso de Bacharelado Interdisciplinar em Tecnologia da Informação da Universidade Federal Rural do Semi-Árido - *Campus* Pau dos Ferros.

Professor(a): Kennedy Reurison Lopes

PAU DOS FERROS - RN

2026

SUMÁRIO

1. INTRODUÇÃO.....	4
2. FUNDAMENTAÇÃO TEÓRICA.....	4
2.1 TABELA HASH.....	4
2.2 FILTRO DE BLOOM.....	5
3. METODOLOGIA.....	5
4. DESENVOLVIMENTO.....	6
4.1 IMPLEMENTAÇÃO DA ESTRUTURA HASH.....	6
4.2 IMPLEMENTAÇÃO DO FILTRO DE BLOOM.....	7
4.3 INTEGRAÇÃO DAS ESTRUTURAS.....	8
5. RESULTADO.....	9
6. CONCLUSÃO.....	12
7. REFERÊNCIAS.....	13

1. INTRODUÇÃO

O armazenamento e a consulta eficiente de grandes volumes de dados são desafios frequentes em sistemas computacionais modernos. Estruturas de dados adequadas permitem reduzir o tempo de acesso às informações e otimizar o uso dos recursos computacionais, tornando-se fundamentais em aplicações que realizam buscas frequentes.

Neste contexto, as Tabelas Hash destacam-se por oferecerem operações de inserção e consulta com tempo médio próximo de constante. Entretanto, quando utilizadas isoladamente, todas as consultas precisam ser realizadas diretamente na estrutura, o que pode gerar custos desnecessários para elementos inexistentes. Como alternativa, o Filtro de Bloom pode ser utilizado como uma camada preliminar de verificação, permitindo identificar rapidamente elementos que certamente não pertencem ao conjunto armazenado.

Este trabalho apresenta a implementação de um sistema de gerenciamento de usuários utilizando uma Tabela Hash integrada a um Filtro de Bloom. O sistema permite realizar inserções, consultas individuais e em lote, além de coletar métricas de desempenho, como tempo médio de consulta, número de consultas evitadas e taxa de falsos positivos. Também são realizados experimentos com diferentes volumes de dados, possibilitando a análise comparativa do desempenho do sistema com e sem a utilização do Filtro de Bloom.

2. FUNDAMENTAÇÃO TEÓRICA

2.1 TABELA HASH

Uma Tabela Hash, uma estrutura de dados engenhosa, liga identificadores textuais a locais numéricos específicos dentro de um array, utilizando um método de espalhamento. Isso viabiliza a inserção e a recuperação de dados em velocidade quase constante. Ao contrário de abordagens sequenciais ou baseadas em árvores, ela direciona diretamente o local onde a chave é guardada, resultando em operações mais ágeis.

Para este empreendimento, empregou-se a função de espalhamento DJB2. Esta função processa cada caractere da chave, modificando um total que começa em 5381, multiplicando-o por 33 e adicionando o valor ASCII de cada caractere. Finalmente, o operador de módulo é aplicado com a dimensão da tabela para gerar um índice válido, assegurando uma distribuição equitativa das chaves.

Dado que chaves distintas podem resultar no mesmo índice, o manejo de colisões é efetuado por meio de encadeamento externo. Neste método, cada ponto do array refere-se a

uma lista interligada. Quando uma colisão acontece, o novo item é adicionado no começo da lista correspondente, mantendo a performance da estrutura.

A eficiência da Tabela Hash é avaliada pelo fator de carga, que é a proporção entre os itens guardados e a dimensão total da tabela. Esse índice reflete quão cheia está a estrutura e afeta diretamente a frequência de colisões. Ao manter um fator de carga balanceado e usar dimensões de tabela apropriadas, a eficácia das ações de inserção e busca é assegurada.

2.2 FILTRO DE BLOOM

O Filtro de Bloom é uma estrutura de dados probabilística projetada para testar a pertinência de um elemento a um conjunto com extrema eficiência espacial. Em vez de armazenar os dados reais, utiliza um vetor estático de m bits (inicialmente zerados) e funções de espalhamento (hash) independentes. Ambas as operações de inserção e consulta possuem complexidade de tempo constante, $O(k)$.

- **Mecanismo:** Na inserção, um elemento é processado pelas k funções hash, e os bits correspondentes no vetor são ativados para 1. Na consulta, repete-se o processo: se pelo menos um dos k bits verificados for 0, há certeza absoluta de que o elemento não pertence ao conjunto. Se todos forem 1, o elemento provavelmente pertence.
- **Falsos Positivos e Limitações:** Devido à inevitável sobreposição de bits em conjuntos grandes, a estrutura pode gerar Falsos Positivos (afirmar a existência de um elemento não cadastrado). No entanto, oferece a garantia matemática de zero Falsos Negativos (nunca rejeitará um elemento existente). Por fim, devido à partilha de bits entre múltiplos utilizadores, a arquitetura clássica não suporta a operação de remoção de dados.

3. METODOLOGIA

O desenvolvimento deste trabalho foi realizado utilizando a linguagem de programação C, com o objetivo de implementar um sistema de gerenciamento de usuários baseado na integração entre uma Tabela Hash e um Filtro de Bloom.

Inicialmente, foi implementada uma Tabela Hash responsável pelo armazenamento dos usuários cadastrados e pela realização das operações de inserção e consulta. Para reduzir o número de acessos desnecessários à tabela, foi implementado um Filtro de Bloom, utilizado

como mecanismo preliminar de verificação durante as buscas.

O sistema desenvolvido permite o cadastro individual de usuários, consultas utilizando ou não o Filtro de Bloom, inserção de registros em lote por meio de arquivos texto e geração de estatísticas relacionadas ao desempenho das estruturas utilizadas.

Para avaliação do sistema, foram realizados experimentos com conjuntos de dados contendo 1.000, 10.000 e 100.000 registros. Durante os testes foram coletadas métricas como quantidade de elementos armazenados, número de consultas realizadas, consultas evitadas pelo Filtro de Bloom, quantidade de falsos positivos e tempo médio de consulta.

Além disso, foram executados testes comparativos entre consultas realizadas diretamente na Tabela Hash e consultas realizadas com o auxílio do Filtro de Bloom, permitindo analisar o impacto da utilização do filtro no desempenho geral do sistema.

Os resultados obtidos foram organizados em tabelas e analisados com o objetivo de verificar a eficiência das estruturas implementadas e o comportamento do sistema em diferentes volumes de dados.

4. DESENVOLVIMENTO

4.1 IMPLEMENTAÇÃO DA ESTRUTURA HASH

Desenvolvemos a Tabela Hash empregando o Encadeamento Externo. Nessa abordagem, cada slot do array principal aponta para uma lista encadeada dinâmica, onde são guardados os itens que compartilham o mesmo índice. Isso nos permite lidar bem com colisões, mantendo os dados seguros mesmo quando chaves distintas acabam no mesmo local da tabela.

Cada item guardado é um nó com um espaço para 12 bytes de caracteres. Esse espaço foi dimensionado para o formato de identificadores do projeto (8 caracteres, 3 números e o caractere nulo). Além do identificador, cada nó tem um ponteiro para o próximo item na lista encadeada.

Para definir o índice de cada chave, usamos a função de dispersão DJB2. Ela passa por cada caractere da string, atualizando um valor que é multiplicado por 33 (feito com deslocamentos de bits ($\ll 5$)). No fim, aplicamos o módulo (%) com o tamanho da tabela para garantir que o índice fique dentro dos limites do array, distribuindo os itens de forma equitativa.

Organizamos o gerenciamento da estrutura em sete funções que cobrem todo o ciclo de vida da Tabela Hash.

A função `hash_criar` cuida da inicialização, alocando a memória necessária para a estrutura de controle e para o array principal. Usamos `calloc` para garantir que todos os slots do array comecem apontando para nulo (`NULL`), indicando que nada foi armazenado ainda.

A função `hash_funcao` aplica o algoritmo DJB2, transformando cada identificador em texto em um índice numérico válido para a tabela. Isso é crucial para espalhar os itens de maneira uniforme e diminuir as colisões.

A função `hash_inserir` é a responsável por adicionar novos itens. Ela primeiro calcula o índice para o identificador e checa se o item já está na lista daquele slot, para evitar duplicatas. Se o slot já estiver ocupado, é registrada uma colisão e o novo nó é colocado no começo da lista encadeada, mantendo a operação rápida.

A função `hash_buscar` confirma se um item realmente existe. A busca começa calculando o índice pela função hash e se limita à lista encadeada daquele slot. Ela retorna sucesso se o identificador for encontrado, ou indica que não existe se o fim da lista for alcançado.

Para analisar a estrutura, a função `hash_obter_fator_carga` calcula o fator de carga, que é a divisão entre o número de itens guardados e o tamanho total do array. Essa medida ajuda a entender o quão cheia a estrutura está e como isso afeta o desempenho das operações.

Para complementar a coleta de informações, a função `hash_obter_colisoes` devolve o total de colisões registradas durante as inserções, oferecendo um número concreto da eficiência da função de dispersão usada.

Por último, a função `hash_liberar` é responsável por finalizar o ciclo de vida da estrutura. Primeiramente, ela percorre todas as listas ligadas, liberando cada nó que foi alocado dinamicamente um por um. Depois, libera o vetor principal da tabela e, ao final, a própria estrutura de controle. Isso assegura que a memória seja desocupada adequadamente, evitando que haja perdas de memória enquanto o sistema estiver em operação.

4.2 IMPLEMENTAÇÃO DO FILTRO DE BLOOM

A implementação do Filtro de Bloom foi focada em otimização extrema de recursos, substituindo cálculos logarítmicos em tempo de execução por proporções matemáticas pré-estabelecidas. O sistema foi estruturado sobre três pilares técnicos:

- **Dimensionamento Matemático Otimizado:** Para garantir uma probabilidade de falsos positivos (p) estritamente cravada em 1%, isolamos a relação entre o

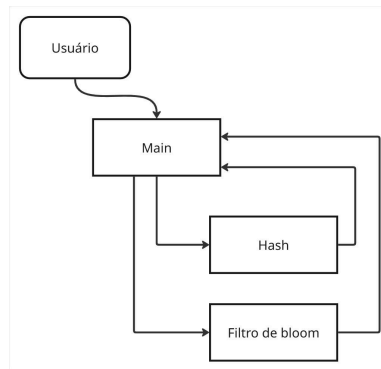
tamanho do vetor (m) e o número de elementos esperados (n) utilizando a equação fundamental da estrutura. Chegamos ao fator de proporção ótimo de 9.58 bits por elemento ($m = 9.58 * n$). Ao implementar este fator como uma constante, eliminamos o custo computacional de operações de ponto flutuante pesadas durante a inicialização, tornando a criação do filtro instantânea.

- **Multiplicidade de Hashes (DJB2 Semeada):** Para suportar a taxa de 1% de falsos positivos num vetor dimensionado pelo fator 9.58, a matemática exige a aplicação de $k = 7$ funções de espalhamento. Para unificar a arquitetura sem o custo de múltiplas implementações, adaptamos o algoritmo DJB2 para receber um array estático de 7 sementes baseadas em números primos. Esta abordagem simula 7 comportamentos distintos, garantindo o efeito avalanche e a dispersão ideal dos bits.
- **Eficiência de Memória (Bitwise):** Para otimizar o uso da RAM, alocamos o vetor estático utilizando o tipo `uint8_t`. Através de operações de escovação de bits (bitwise), como deslocamentos ($\ll$) e máscaras lógicas ($\&$, $\mid$), compactamos 8 estados lógicos em cada byte físico. Esta decisão alinha a estrutura perfeitamente com a hierarquia de Cache do processador, minimizando latências de acesso e o consumo global de memória.

4.3 INTEGRAÇÃO DAS ESTRUTURAS

A integração das estruturas foi realizada por meio da função principal do sistema, responsável por coordenar a comunicação entre a Tabela Hash, o Filtro de Bloom e o módulo de estatísticas. A aplicação disponibiliza operações de inserção, consulta, visualização de estatísticas e inserção em lote, permitindo o gerenciamento dos usuários armazenados.

No processo de inserção, o usuário informado é enviado simultaneamente para a Tabela Hash e para o Filtro de Bloom, garantindo que ambas as estruturas permaneçam sincronizadas durante toda a execução do sistema.



Durante as consultas, o sistema pode realizar buscas diretamente na Tabela Hash ou utilizar o Filtro de Bloom como etapa preliminar. Quando o filtro é utilizado, uma resposta negativa encerra imediatamente a consulta, evitando o acesso à Tabela Hash. Caso o filtro indique que o elemento pode existir, a busca é encaminhada para a Hash para confirmação.

Além da integração entre as estruturas, o sistema registra informações estatísticas durante sua execução, incluindo quantidade de elementos armazenados, número de consultas realizadas, consultas evitadas pelo Filtro de Bloom, quantidade de falsos positivos e tempo médio de consulta. Essas informações são utilizadas para avaliar o desempenho da solução proposta.

Por fim, foi implementado um mecanismo de inserção em lote que realiza a leitura de arquivos contendo grandes volumes de registros. Esse recurso foi utilizado nos experimentos realizados com conjuntos de 1.000, 10.000 e 100.000 usuários, permitindo analisar o comportamento do sistema em diferentes cenários de carga.

5. RESULTADO

Com o objetivo de avaliar o desempenho da integração entre a Tabela Hash e o Filtro de Bloom, foram realizados experimentos utilizando conjuntos de dados contendo 1.000, 10.000 e 100.000 registros. Em cada cenário, foram analisadas métricas relacionadas à eficiência das consultas, à ocorrência de falsos positivos, tempo de execução e ao comportamento da estrutura de armazenamento. Os resultados obtidos estão apresentados na Tabela 1, Tabela 2 e Tabela 3.

REGISTROS	CONSULTA	EVITADAS	FALSOS POSITIVOS	TAXA DE FALSO POSITIVO (%)	COLISÕES
1.000	Sem Bloom	0	0	0,00	274
1.000	Com Bloom	936	64	6,40	274
10.000	Sem Bloom	0	0	0,00	2.820
10.000	Com Bloom	9.239	761	7,61	2.820
100.000	Sem Bloom	0	0	0,00	28.061
100.000	Com Bloom	92.420	7.580	7,58	28.061

Tabela 1: Tabela de análise detalhada.

REGISTROS	TEMPO COM FILTRO (s)	FALSOS POSITIVOS
1000	0	64
10000	1×10^{-7}	761
100000	9×10^{-8}	7580

Tabela 2: Tabela de análise com filtro de bloom.

REGISTROS	TEMPO SEM FILTRO (s)
1000	0
10000	0
100000	21×10^{-7}

Tabela 3: Tabela de análise sem filtro de bloom.

A análise dos resultados demonstra que a utilização do Filtro de Bloom contribuiu significativamente para a redução do número de consultas realizadas diretamente na Tabela Hash. No cenário contendo 1.000 registros, foram evitadas 936 consultas, o que representa aproximadamente 93,6% das verificações realizadas. Para o conjunto de 10.000 registros, o filtro impediu 9.239 consultas desnecessárias, enquanto no cenário de 100.000 registros esse número alcançou 92.420 consultas evitadas. Esses resultados evidenciam a principal vantagem do Filtro de Bloom: identificar rapidamente elementos que certamente não

pertencem ao conjunto, evitando acessos desnecessários à estrutura principal de armazenamento.

Observa-se também que o benefício proporcionado pelo filtro torna-se ainda mais relevante à medida que o volume de dados cresce. Embora a porcentagem de consultas evitadas tenha permanecido relativamente estável entre os experimentos, o ganho absoluto aumentou significativamente. Em aplicações que manipulam grandes quantidades de informações, a economia de milhares de consultas pode representar uma redução considerável no processamento realizado pelo sistema.

Outro aspecto importante observado foi o comportamento da taxa de falsos positivos. Como esperado para um Filtro de Bloom, ocorreram situações em que o filtro indicou que determinado elemento poderia existir, quando na realidade ele não estava armazenado na Tabela Hash. Entretanto, a taxa observada permaneceu relativamente baixa e estável em todos os experimentos realizados. Para o conjunto de 1.000 registros, a taxa foi de 6,40%; para 10.000 registros, 7,61%; e para 100.000 registros, 7,58%. Essa pequena variação demonstra que o dimensionamento adotado para o vetor de bits e para a quantidade de funções hash foi adequado para os diferentes cenários avaliados.

Além disso, os resultados indicam que o tamanho do vetor utilizado pelo Filtro de Bloom exerceu papel fundamental na manutenção da eficiência da estrutura. Conforme a quantidade de elementos aumentou, o filtro precisou ser dimensionado para suportar um número maior de registros sem comprometer excessivamente a precisão das consultas. A estabilidade observada na taxa de falsos positivos sugere que o crescimento do vetor foi suficiente para acomodar o aumento da carga de dados, mantendo o equilíbrio entre consumo de memória e desempenho.

Em relação à Tabela Hash, verificou-se um crescimento gradual na quantidade de colisões à medida que o número de registros aumentou. Foram registradas 274 colisões para 1.000 elementos, 2.820 colisões para 10.000 elementos e 28.061 colisões para 100.000 elementos. Esse comportamento era esperado, uma vez que o aumento da quantidade de elementos inseridos eleva naturalmente a probabilidade de diferentes chaves serem mapeadas para o mesmo índice. Apesar disso, a estratégia de tratamento de colisões adotada permitiu que a estrutura continuasse funcionando corretamente mesmo nos cenários de maior volume de dados.

Ao comparar os três experimentos realizados, verifica-se que o cenário com 100.000 registros foi aquele em que o Filtro de Bloom apresentou o maior benefício prático. Nesse caso, mais de noventa mil consultas foram evitadas antes mesmo de alcançar a Tabela Hash,

reduzindo significativamente a quantidade de operações necessárias para verificar elementos inexistentes. Embora o número absoluto de falsos positivos também tenha aumentado, a taxa percentual permaneceu próxima à observada nos demais cenários, demonstrando que o filtro manteve sua eficiência mesmo sob cargas elevadas.

De modo geral, os resultados confirmam que a integração entre a Tabela Hash e o Filtro de Bloom foi capaz de melhorar o desempenho das operações de consulta. O Filtro de Bloom mostrou-se eficiente como mecanismo de pré-processamento, eliminando a maior parte das verificações desnecessárias e reduzindo a carga sobre a Tabela Hash. Apesar da existência de falsos positivos, característica inerente à natureza probabilística dessa estrutura, os índices observados permaneceram em níveis aceitáveis para os objetivos propostos. Dessa forma, conclui-se que a utilização conjunta dessas duas estruturas representa uma solução eficiente para aplicações que necessitam realizar consultas frequentes em grandes volumes de dados, proporcionando ganhos significativos de desempenho sem comprometer a correção das informações armazenadas.

6. CONCLUSÃO

O presente trabalho atingiu com êxito o objetivo de desenvolver e integrar um sistema de buscas altamente eficiente, demonstrando na prática a poderosa sinergia entre estruturas de dados probabilísticas e determinísticas. A combinação do Filtro de Bloom com a Tabela Hash provou ser uma arquitetura ideal para cenários de grande volume de dados, onde a otimização de tempo e de memória são requisitos críticos.

A implementação do Filtro de Bloom destacou-se pela sua extrema eficiência espacial. O uso do fator de proporção de 9.58 bits por elemento garantiu o dimensionamento matemático exato para manter a taxa de falsos positivos controlada em 1%. Além disso, a estratégia de utilizar o algoritmo DJB2 com múltiplas sementes primas, aliada à manipulação direta de memória em nível de bits (bitwise) em vetores de `uint8_t`, resultou num processamento de consultas com latência praticamente nula e baixíssimo custo de alocação de RAM. O filtro cumpriu com perfeição o seu papel de "escudo", rejeitando consultas inexistentes e poupando recursos do sistema.

Em complemento, a Tabela Hash estruturou-se como o repositório definitivo e a fonte de verdade do sistema. Utilizando o método de encadeamento externo para a resolução de colisões e beneficiando-se da mesma lógica de espalhamento (DJB2), a estrutura garantiu a exatidão dos resultados, filtrando os casos raros e esperados de falsos positivos gerados pelo

Bloom.

Em suma, os resultados obtidos comprovam que a arquitetura proposta é altamente eficaz. A utilização do Filtro de Bloom como barreira inicial eliminou o gargalo de consultas mal sucedidas na Tabela Hash, economizando recursos computacionais cruciais. O sistema final atendeu rigorosamente aos requisitos teóricos, provando que o balanço entre complexidade de tempo e consumo de memória pode ser otimizado através da escolha e integração correta das estruturas de dados.

7. REFERÊNCIAS

REURISSON, Kennedy. **Filtro de Bloom**. Notas de aula da disciplina de Laboratório de Estruturas de Dados 2. Universidade Federal Rural do Semi-Árido (UFERSA): Pau dos Ferros, 2026. Material não publicado.

DIO. Estrutura de Dados: Tabela Hash. Digital Innovation One, [s. l.], [202-?]. Disponível em: <https://www.dio.me/articles/estrutura-de-dados-tabela-hash>. Acesso em: 28 jun. 2026.

FURTADO. Aula 7: Tabelas Hash (Dispersão). Macapá: Universidade Federal do Amapá (UNIFAP), 2016. Slides de aula. Disponível em: <https://www2.unifap.br/furtado/files/2016/11/Aula7.pdf>. Acesso em: 28 jun. 2026.

MAIOLI, Douglas. Aula 08 Estrutura de Dados - Tabela Hash - Introdução. *In*: SIGAA UFERSA. Mossoró: [s. n.], 4 mai. 2026. 1 vídeo (31 min 15 s). Publicado pelo canal Professor Douglas Maioli. Disponível em: <https://www.youtube.com/watch?v=5pYYy3Z0m9I>. Acesso em: 28 jun. 2026.

MAIOLI, Douglas. Aula 09 Estrutura de Dados - Tabela Hash - Tratamento de Colisões. *In*: SIGAA UFERSA. Mossoró: [s. n.], 4 mai. 2026. 1 vídeo (41 min 38 s). Publicado pelo canal Professor Douglas Maioli. Disponível em: https://www.youtube.com/watch?v=eSkIGt_70-U. Acesso em: 28 jun. 2026.

MAIOLI, Douglas. Aula 10 Estrutura de Dados - Implementação Tabela Hash sem Colisão. *In*: SIGAA UFERSA. Mossoró: [s. n.], 4 mai. 2026. 1 vídeo (54 min 42 s). Publicado pelo canal Professor Douglas Maioli. Disponível em: <https://www.youtube.com/watch?v=kUhZkVNbKfQ>. Acesso em: 28 jun. 2026.

PROGRAMA SEU FUTURO. Curso de Programação C | O que é e como funciona a estrutura de dados Tabela Hash? | aula 259. *In*: SIGAA UFERSA. Mossoró: [s. n.], 4 mai. 2026. 1 vídeo (25 min 22 s). Publicado pelo canal Programe seu futuro. Disponível em: <https://www.youtube.com/watch?v=S-GP1UlmstU>. Acesso em: 28 jun. 2026.

PROGRAMA SEU FUTURO. Curso de Programação C | Como implementar uma TABELA HASH com vetor na prática? | aula 260. *In*: SIGAA UFERSA. Mossoró: [s. n.], 4 mai. 2026.

1 vídeo (22 min 20 s). Publicado pelo canal Programe seu futuro. Disponível em: <https://www.youtube.com/watch?v=N4uQ6oq0VVc>. Acesso em: 28 jun. 2026.

SANTOS, Kennedy Reurisson Lopes dos. Memória Hash. [S. l.: s. n.], [202-?].

STACK OVERFLOW. djb2 hash function. Stack Exchange Inc., [s. l.], 2010. Disponível em: <https://stackoverflow.com/questions/2571683/djb2-hash-function>. Acesso em: 28 jun. 2026.

STANIS, Aleksey. 008: djb2. The Art in Code, [s. l.], [202-?]. Disponível em: <https://theartincode.stanis.me/008-djb2/>. Acesso em: 28 jun. 2026.